

TRANSPORT LAYER

Contents:Transport Layer Services, UDP: Datagram, Services, Applications, TCP: Services, Features, Segment, TCP Connection, Window in TCP, Flow control, Congestion Control, Congestion Control Algorithms, Leaky Bucket, Token Bucket and QoS, TCP Timers, Options, TCP Package, Applications, SCTP: Features, Services, Packet Format, Socket: TCP and UDP Socket, Applications

***Unit Objectives and outcomes:** On completion the students will be able to:*

To introduce transport layer services and flow control and congestion control mechanism.

Unit outcomes : On completion the students will be able to
Understand services and protocols used at transport layer.

Outcome Mapping:

PEOs:

POs:

COs: CO6

PSOs:

Transport Layer

What is Transport Layer?

The Transport Layer is responsible for end-to-end delivery over a network. Whereas the network layer is concerned with the end - to- end delivery of individual packets and it does not recognize any relationship between those packets.

- This layer treats each packet independently because each packet belongs to a different message.
- The transport layer ensures that each message should reach its destination completely and in order so that it maintains error and flow control to the source to destination to ensure proper data transmission.
- The transport layer establishes a connection between two end ports. A connection is a single logical path from source to destination which is associated with all the packets in a message.
- Transport Layer uses some standard protocols to enhance its functionalities are TCP(Transmission Control Protocol), UDP(User Datagram Protocol), DCCP(Datagram Congestion Control Protocol), etc.

functions of the transport layer are as follows:

- Efficient delivery of data with proper correction.
- From the technological changes separate all upper layers.
- Managing error control and flow control.
- Obtaining data from the Session layer, divide it into segments and transmit to the network layer.

Transport Layer Services

1. Service-point addressing

Computers often run many programs at the same time. Due to this, source-to-destination delivery means delivery from a specific job (currently running program) on one computer to a specific job (currently running program) on the other system not only one computer to the next. For this reason, the **transport layer** added a specific type of address to its header, it is referred to as a service point address or port address.

2. Segmentation and Reassembly

In segmentation, a message is divided into transmittable segments; each segment containing a sequence number. This number enables this layer to reassemble the message.

Upon arriving at its destination system message is reassembled correctly, identify and replaces packets that were lost in transmission.

3. Connection Control

It can be either of two types:

Connectionless Transport Layer

Connection Oriented Transport Layer

i) Connectionless Transport Layer

This Transport Layer treats each packet as an individual and delivers it to the destination machine.

In this type of transmission, the receiver does not send an acknowledgment to the sender about the receipt of a packet. This is a faster communication technique.

ii) Connection Oriented Transport Layer

This Transport Layer creates a connection with the Transport Layer at the destination machine before transmitting the packets to the destination.

To Create a connection following three steps are possible:

- Connection establishment

- Data transfer

- Connection termination

Connectionless Service is less reliable than connection Oriented Service.

4. Multiplexing and Demultiplexing

The transport layer accepts packets from different processes. These packets are differentiated by their port numbers and pass them to the network layer after adding proper headers.

In Demultiplexing, at the receiver's side to obtain the data coming from various processes. It receives the segments of data from the network layer and delivers it to the appropriate process running on the receiver's machine.

5. Flow control

The transport layer provides a flow control mechanism between the adjacent layers of the TCP/IP model. TCP also prevents data loss due to a fast sender and slow receiver by imposing some flow control techniques. It uses the method of sliding window protocol which is accomplished by the receiver by sending a window back to the sender informing the size of data it can receive.

6. Error Control

Error Control is performed end to end in this layer to ensure that the complete message arrives at the receiving transport layer without any error. Error Correction is done through retransmission.

Transport vs. network layer

Transport Layer	Network Layer
logical communication between <u>processes</u>	logical communication between <u>hosts</u>
exists only in hosts	exists in hosts <u>and</u> in routers
ignores network	routes data through network
Port #s used for routing in destination computer	IP addresses used for routing in network

- Transport layer uses Network layer services
 - adds more value to these services

The User Datagram Protocol (UDP)

The User Datagram Protocol (UDP) is simplest Transport Layer communication protocol available of the TCP/IP protocol suite. It involves minimum amount of communication mechanism. UDP is said to be an unreliable transport protocol but it uses IP services which provides best effort delivery mechanism.

In UDP, the receiver does not generate an acknowledgement of packet received and in turn, the sender does not wait for any acknowledgement of packet sent. This shortcoming makes this protocol unreliable as well as easier on processing.

Features

UDP is used when acknowledgement of data does not hold any significance.

UDP is good protocol for data flowing in one direction.

UDP is simple and suitable for query based communications.

UDP is not connection oriented.

UDP does not provide congestion control mechanism.

UDP does not guarantee ordered delivery of data.

UDP is stateless.

UDP is suitable protocol for streaming applications such as VoIP, multimedia streaming.

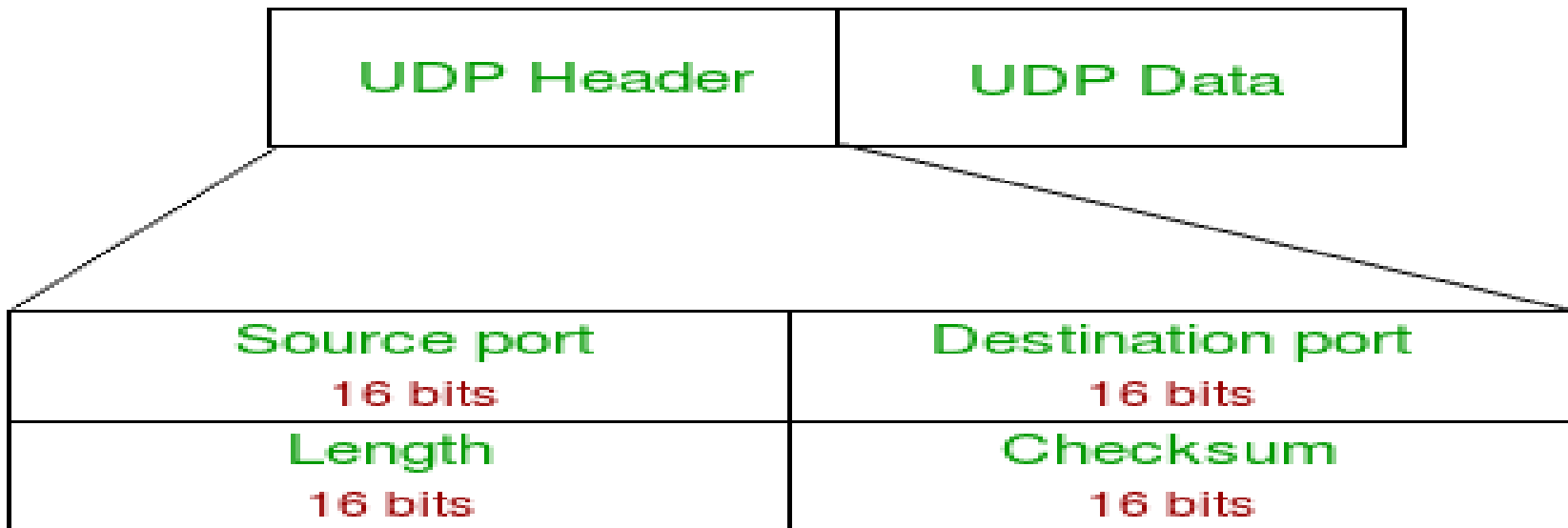
UDP Header –

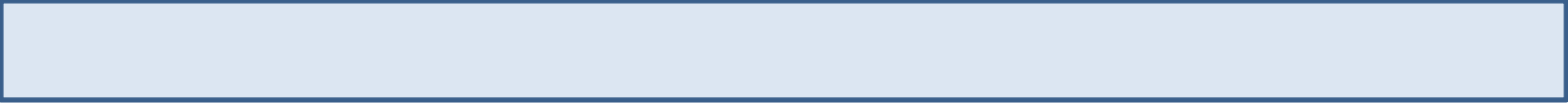
UDP header is **8-bytes** fixed and simple header, while for TCP it may vary from 20 bytes to 60 bytes.

First 8 Bytes contains all necessary header information and remaining part consist of data.

UDP port number fields are each 16 bits long, therefore range for port numbers defined from 0 to 65535; port number 0 is reserved. Port numbers help to distinguish different user requests or process.

8 Bytes





Source Port : Source Port is 2 Byte long field used to identify port number of source.

Destination Port : It is 2 Byte long field, used to identify the port of destined packet.

Length : Length is the length of UDP including header and the data. It is 16-bits field.

Checksum : Checksum is 2 Bytes long field. It is the 16-bit one's complement of the one's complement sum of the UDP header, pseudo header of information from the IP header and the data, padded with zero octets at the end (if necessary) to make a multiple of two octets.

Applications of UDP:

- Used for simple request response communication when size of data is less and hence there is lesser concern about flow and error control.
- It is suitable protocol for multicasting as UDP supports packet switching.
- UDP is used for some routing update protocols like RIP(Routing Information Protocol).
- Normally used for real time applications which can not tolerate uneven delays between sections of a received message.

Transport service primitives

Primitive	TPDU sent	Meaning
Listen	None	Block until some process tries to connect
Connect	Connection request	Actively attempt to establish a connection
Send	Data	Send information
Receive	None	Block until a Data TPDU arrives
Disconnect	Disconnection req.	This side wants to release the connection

Transmission Control Protocol (TCP)

The transmission Control Protocol (TCP) is one of the most important protocols of Internet Protocols suite. It is most widely used protocol for data transmission in communication network such as internet.

Features

- TCP is reliable protocol. That is, the receiver always sends either positive or negative acknowledgement about the data packet to the sender, so that the sender always has bright clue about whether the data packet is reached the destination or it needs to resend it.
- TCP ensures that the data reaches intended destination in the same order it was sent.
- TCP is connection oriented. TCP requires that connection between two remote points be established before sending actual data.
- TCP provides error-checking and recovery mechanism.
- TCP provides flow control and quality of service.
- TCP operates in Client/Server point-to-point mode.
- TCP provides full duplex server, i.e. it can perform roles of both receiver and sender.

TCP Header (Segment)

The length of TCP header is minimum 20 bytes long and maximum 60 bytes.



Source Port (16-bits) - It identifies source port of the application process on the sending device.

Destination Port (16-bits) - It identifies destination port of the application process on the receiving device.

Sequence Number (32-bits) - Sequence number of data bytes of a segment in a session.

Acknowledgement Number (32-bits) - When ACK flag is set, this number contains the next sequence number of the data byte expected and works as acknowledgement of the previous data received.

Data Offset (4-bits) - Data offset specifies the size of the TCP header in 32 bit words. This field implies both, the size of TCP header (32-bit words) and the offset of data in current packet in the whole TCP segment.

Reserved (3-bits) - Reserved for future use and all are set zero by default.

Flags (1-bit each) URG - It indicates that Urgent Pointer field has significant data and should be processed.

ACK - It indicates that Acknowledgement field has significance. If ACK is cleared to 0, it indicates that packet does not contain any acknowledgement.

PSH - When set, it is a request to the receiving station to PUSH data (as soon as it comes) to the receiving application without buffering it.

RST - Reset flag has the following features:

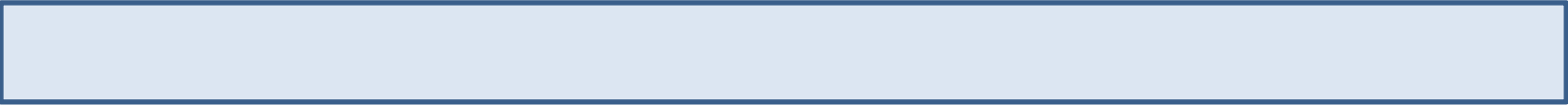
- It is used to refuse an incoming connection.

- It is used to reject a segment.

- It is used to restart a connection.

SYN - This flag is used to set up a connection between hosts.

FIN - This flag is used to release a connection and no more data is exchanged thereafter. Because packets with SYN and FIN flags have sequence numbers, they are processed in correct order.



Windows Size - This field is used for flow control between two stations and indicates the amount of buffer (in bytes) the receiver has allocated for a segment, i.e. how much data is the receiver expecting.

Checksum - This field contains the checksum of Header, Data and Pseudo Headers.

Urgent Pointer - It points to the urgent data byte if URG flag is set to 1.

Options - It facilitates additional options which are not covered by the regular header. Option field is always described in 32-bit words. If this field contains data less than 32-bit, padding is used to cover the remaining bits to reach 32-bit boundary.

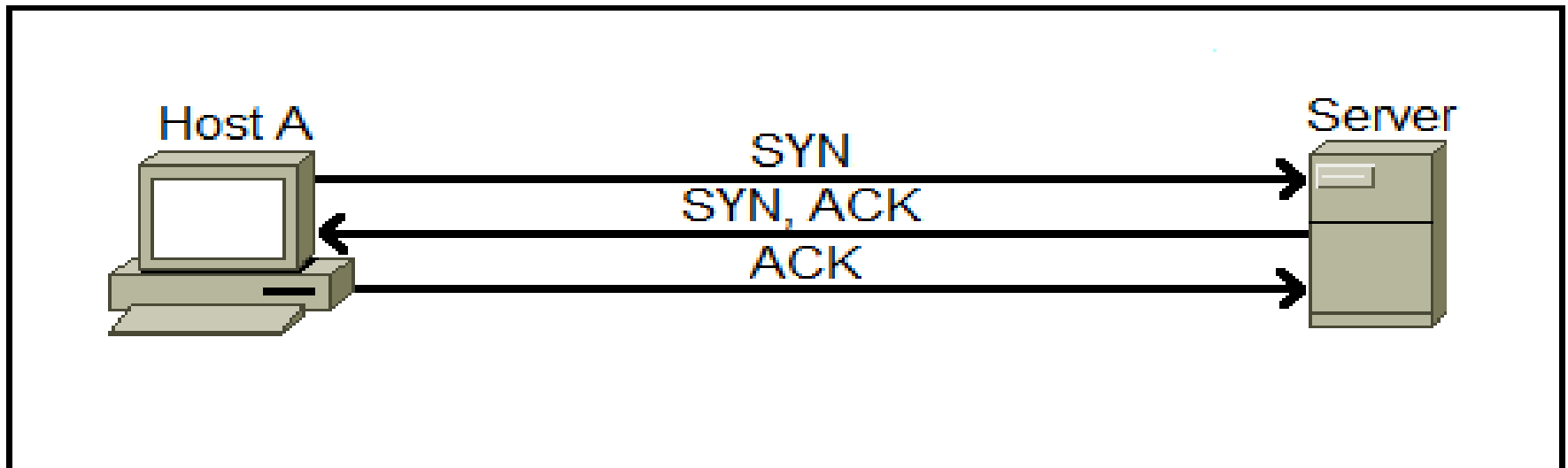
Transmission control protocol (TCP)

- The transmission control protocol (TCP) is the internet standard ensuring the successful exchange of data packets between devices over a network.
- TCP is the underlying communication protocol for a wide variety of applications, including web servers and websites, email applications, FTP and peer-to-peer apps.
- TCP operates with the internet protocol (IP) to specify how data is exchanged online.
- IP is responsible for sending each packet to its destination, while TCP guarantees that bytes are transmitted in the order in which they were sent with no errors or omissions.

TCP three-way handshake

Since TCP is a connection-oriented protocol, a connection needs to be established before two devices can communicate.

TCP uses a process called three-way handshake to negotiate the sequence and acknowledgment fields and start the session. Here is a graphical representation of the process:



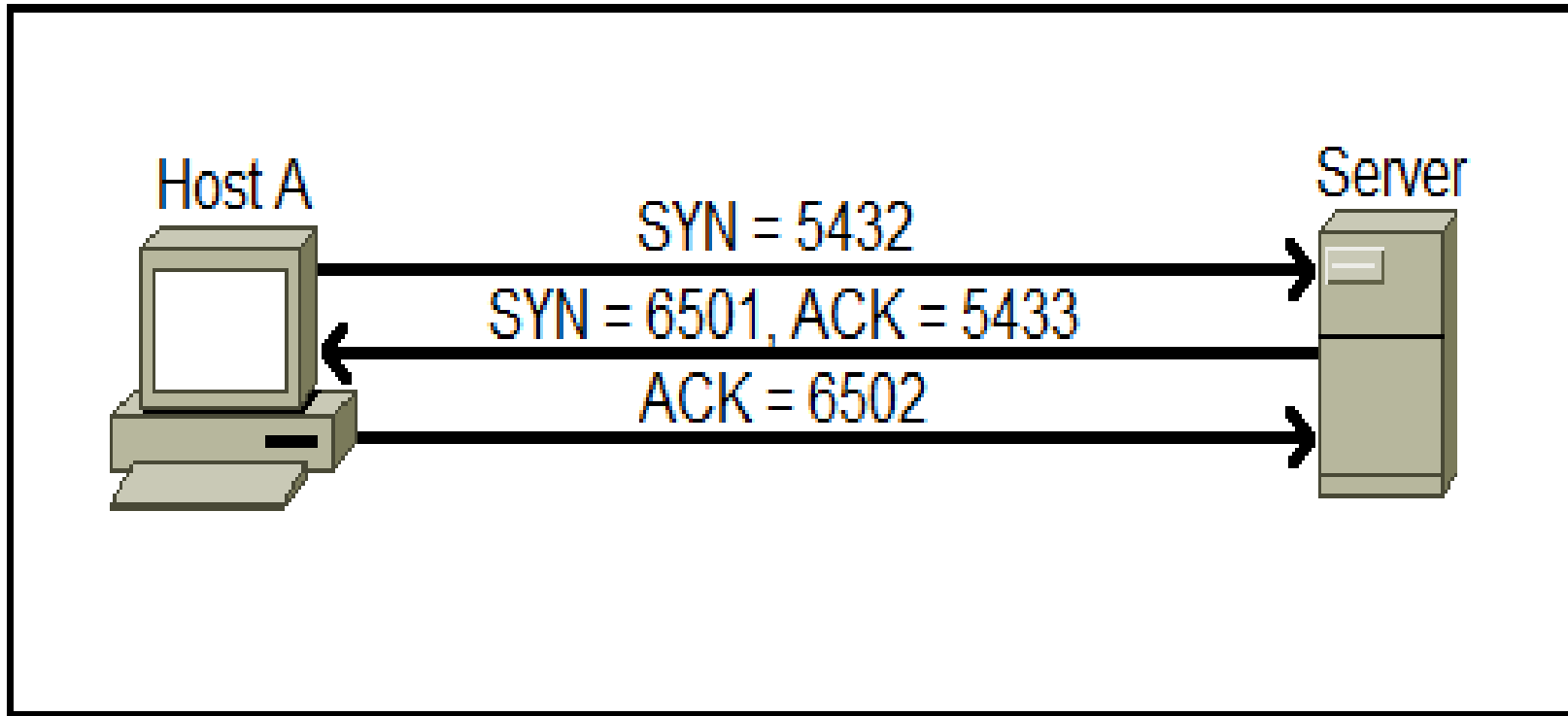
As the name implies, the three way handshake process consists of three steps:

1) Host A initiates the connection by sending the TCP SYN packet to the destination host. The packet contains the random sequence number (e.g. **5432**) which marks the beginning of the sequence numbers for data that the Host A will transmit.

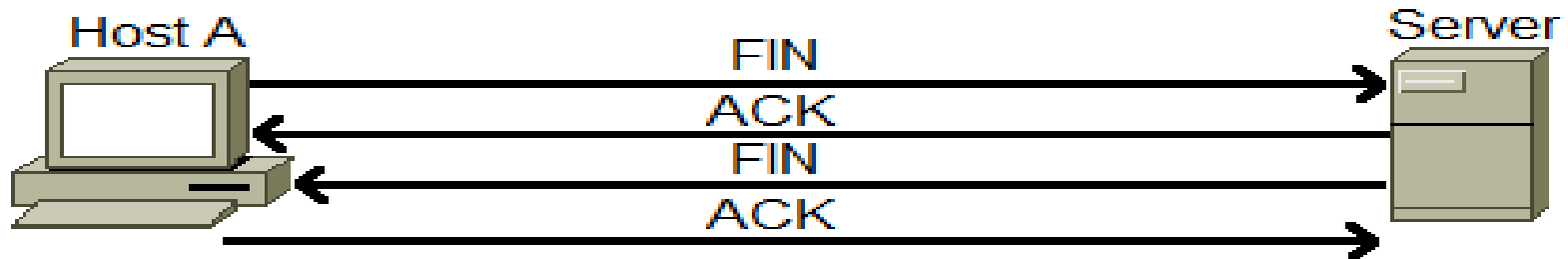
2) The Server receives the packet and responds with its own sequence number. The response also includes the acknowledgment number, which is Host A's sequence number incremented by 1 (in our case, that would be **5433**).

3) Host A acknowledges the response of the Server by sending the acknowledgment number, which is the Server's sequence number incremented by 1.

Here is another picture with the numbers included:



After the data transmission process is finished, TCP will terminate the connection between two endpoints. This four-step process is illustrated below:



- 1) The client application that wants to close the connection sends a TCP segment with the FIN (Finished) flag set to 1.
- 2) The server receives the TCP segment and acknowledges it with the ACK segment.
- 3) Server sends its own TCP segment with the FIN flag set to 1 to the client in order to terminate the connection.
- 4) The client acknowledges the server's FIN segment and closes the connection.

Flow Control

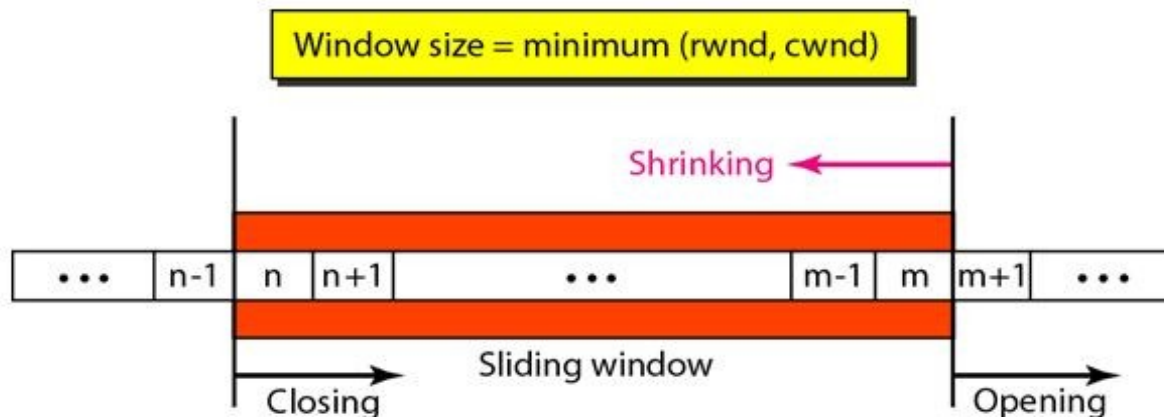
Network Flow Control – regulates the amount of data a source can send before receiving an acknowledgment from the destination

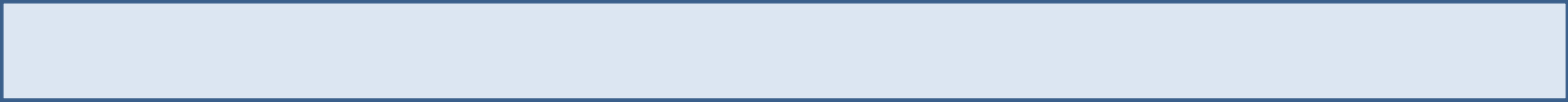
- solution: speed matching service – rate at which sender application is sending is matched against rate at which receiving application is receiving .

TCP uses a sliding window to handle flow control. The sliding window protocol used by TCP, however, is something between the Go-Back-N and Selective Repeat sliding window.

There are two big differences between this sliding window and the one we used at the data link layer. First, the sliding window of TCP is byte-oriented but data link layer sliding window is frame-oriented. Second, the TCP's sliding window is of variable size and the data link layer was of fixed size.

The following figure shows the sliding window in TCP.





The window spans a portion of the buffer containing bytes received from the process. The bytes inside the window are the bytes that can be in transit; they can be sent without worrying about acknowledgment. The imaginary window has two walls: one left and one right.

The window is opened, closed, or shrunk. These three activities, as we will see, are in the control of the receiver (and depend on congestion in the network), not the sender.

Opening a window means moving the right wall to the right. This allows more new bytes in the buffer that are eligible for sending.

Closing the window means moving the left wall to the right. This means that some bytes have been acknowledged and the sender need not worry about them anymore.

Shrinking the window means moving the right wall to the left. This is not allowed in some implementations because it means revoking the eligibility of some bytes for sending.

Features of TCP sliding window are as follows:

- The size of the window is the lesser of rwnd (receiver window) and cwnd (Congestion window)
- The source does not have to send a full window's worth of data.
- The window can be opened or closed by the receiver, but should not be shrunk.
- The destination can send an acknowledgment at any time as long as it does not result in a shrinking window.
- The receiver can temporarily shut down the window; the sender, however, can always send a segment of 1 byte after the window is shut down.

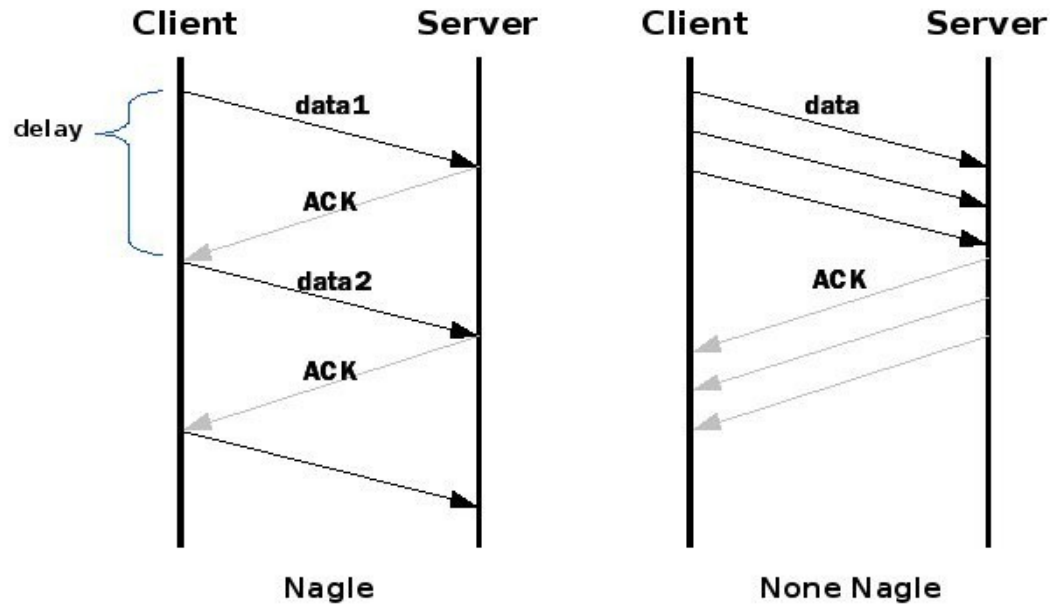
What is Nagle's algorithm?

Nagle's algorithm is an algorithm used in implementations of TCP/IP that controls traffic congestion on a network.

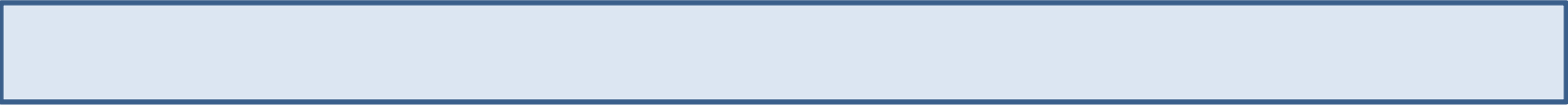
Nagle's algorithm limits transmission of small datagrams and controls the size of the Transmission Control Protocol (TCP) sending window.

The algorithm increases the efficiency of routers by reducing the latency of the routing process.

It is also an important mechanism for enabling large internetworks such as the Internet to handle TCP/IP applications (such as Telnet applications) that communicate using large numbers of small Internet Protocol (IP) packets.

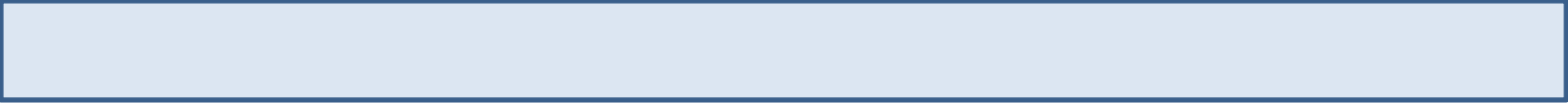


For example, consider a Telnet client communicating with a Telnet server. When a user types a single character in the Telnet client window, TCP packages the character and sends it to the server as a very small packet. It would be highly inefficient if each character typed into a Telnet window were sent as an individual packet. In fact, when the user types the next character, one of two things can happen:



➤ If an acknowledgment has been received for the first packet sent, TCP packages the second character into a second small packet and sends it immediately to prevent delay.

➤ If an acknowledgment has not been received for the first packet, TCP buffers the second character, then the third, and so on until the acknowledgment is received or the buffer has reached a certain size, triggering TCP to package the buffered characters and send them.



This is a result of Nagle's algorithm being implemented on the client TCP/IP protocol stack.

The process of buffering the TCP send buffer is sometimes called “nagling.”

Nagle's algorithm works by combining a number of small outgoing messages and sending them all at once.

Specifically, as long as there is a sent packet for which the sender has received no acknowledgment, the sender should keep buffering its output until it has a full packet's worth of output, thus allowing output to be sent all at once.

TCP Timers

uses several timers to ensure that excessive delays are not encountered during communications. Several of these timers are elegant, handling problems that are not immediately obvious at first analysis. Each of the timers used by TCP is examined in the following sections, which reveal its role in ensuring data is properly sent from one connection to another.



Time Out Timer-(TCP uses a time out timer for retransmission of lost segments.)

- Sender starts a time out timer after transmitting a TCP segment to the receiver.
- If sender receives an acknowledgement before the timer goes off, it stops the timer.
- If sender does not receives any acknowledgement and the timer goes off, then TCP Retransmission occurs.
- Sender retransmits the same segment and resets the timer.
- The value of time out timer is dynamic and changes with the amount of traffic in the network.
- Time out timer is also called as **Retransmission Timer**.

Time Wait Timer- (TCP uses a time wait timer during connection termination.)

- Sender starts the time wait timer after sending the ACK for the second FIN segment.
- It allows to resend the final acknowledgement if it gets lost.
- It prevents the just closed port from reopening again quickly to some other application.
- It ensures that all the segments heading towards the just closed port are discarded.
- The value of time wait timer is usually set to twice the lifetime of a TCP segment.

Keep Alive Timer-(TCP uses a keep alive timer to prevent long idle TCP connections.)

Each time server hears from the client, it resets the keep alive timer to 2 hours.

If server does not hear from the client for 2 hours, it sends 10 probe segments to the client.

These probe segments are sent at a gap of 75 seconds.

If server receives no response after sending 10 probe segments, it assumes that the client is down.

Then, server terminates the connection automatically.

Persistent Timer-

TCP uses a persistent timer to deal with a zero-widow-size deadlock situation.

It keeps the window size information flowing even if the other end closes its receiver window.

Congestion in Network-

A state occurring in network layer when the message traffic is so heavy that it slows down network response time.

Congestion is an important issue that can arise in **Packet Switched Network**.

Congestion leads to the loss of packets in transit. So, it is necessary to control the congestion in network. it is not possible to completely avoid the congestion.

Congestion Control-

Congestion control refers to techniques and mechanisms that can-

Either prevent congestion before it happens

Or remove congestion after it has happened

TCP Congestion Control-

TCP reacts to congestion by reducing the sender window size.

The size of the sender window is determined by the following two factors-

Receiver window size

Congestion window size.

1. Receiver Window Size-

Receiver window size is an advertisement of-

“How much data (in bytes) the receiver can receive without acknowledgement?”

Sender should not send data greater than receiver window size.

Otherwise, it leads to dropping the TCP segments which causes **TCP Retransmission**

So, sender should always send data less than or equal to receiver window size.

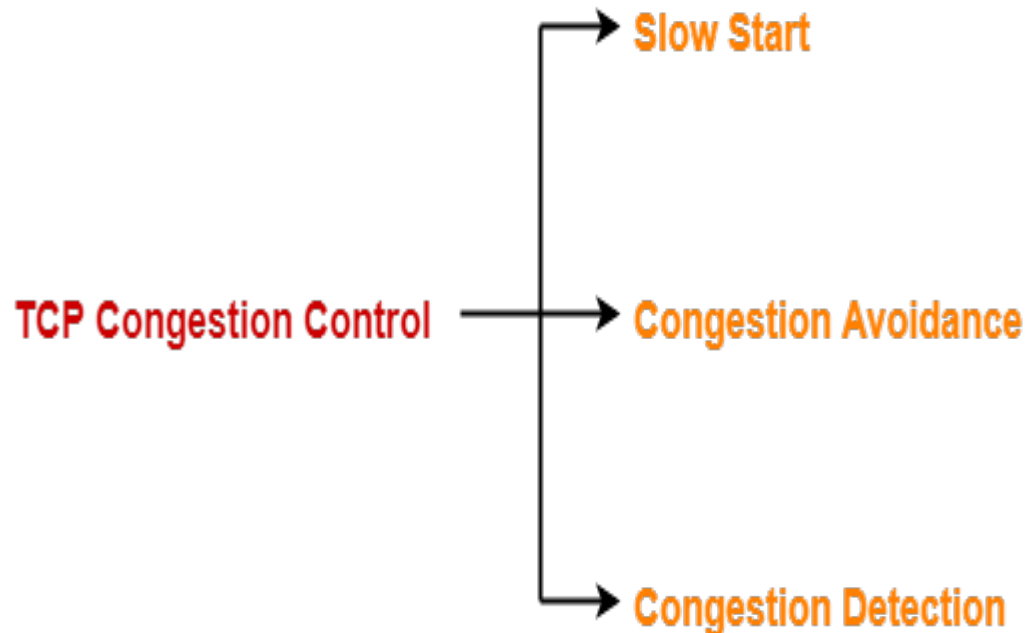
Receiver dictates its window size to the sender through **TCP Header**

2. Congestion Window-

Sender should not send data greater than congestion window size. Otherwise, it leads to dropping the TCP segments which causes TCP Retransmission. So, sender should always send data less than or equal to congestion window size. Different variants of TCP use different approaches to calculate the size of congestion window. Congestion window is known only to the sender and is not sent over the links. So, always- **Sender window size = Minimum (Receiver window size, Congestion window size)**

TCP Congestion Policy-

TCP's general policy for handling congestion consists of following three phases-



Slow Start

Congestion Avoidance

Congestion Detection

Slow Start Phase-

Initially, sender sets congestion window size = Maximum Segment Size (1 MSS).

After receiving each acknowledgment, sender increases the congestion window size by 1 MSS.

In this phase, the size of congestion window increases exponentially.

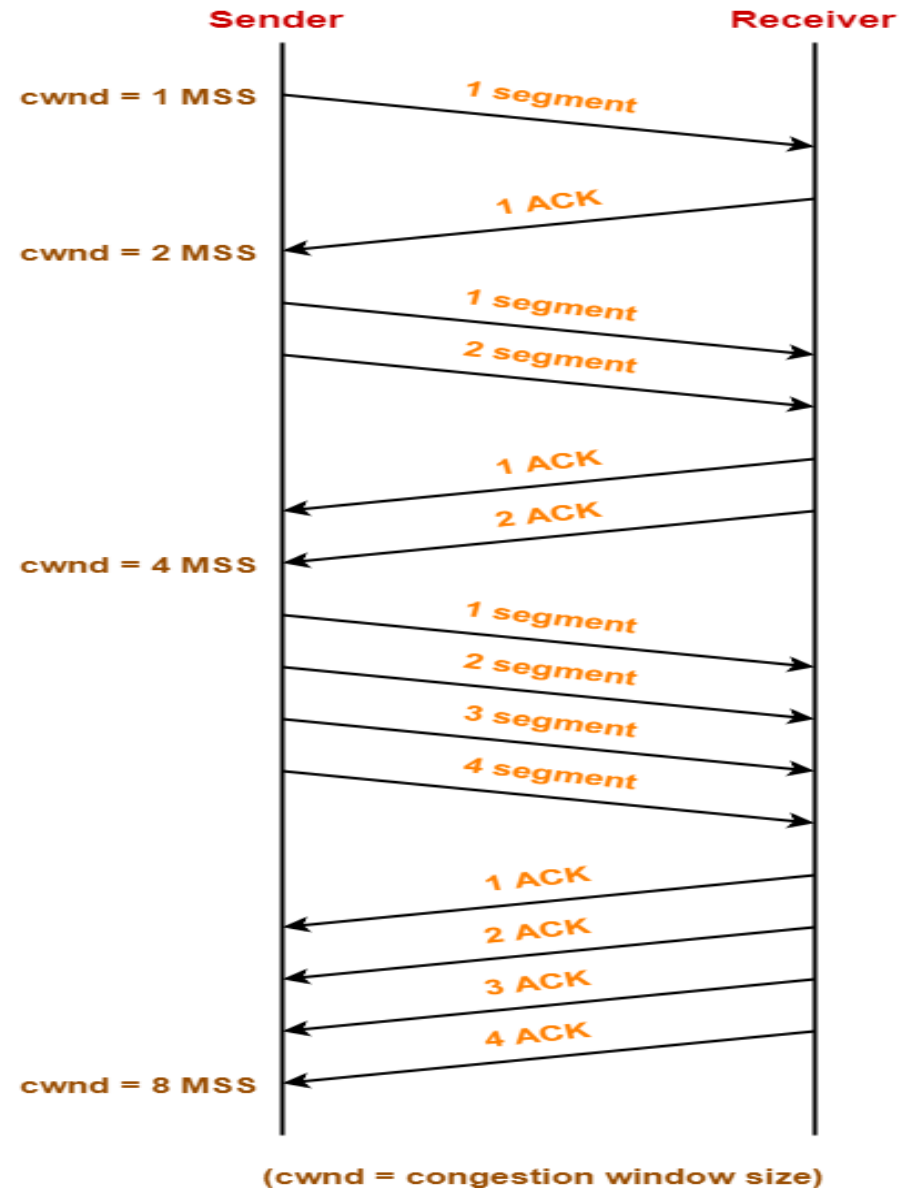
The followed formula is-

Congestion window size = Congestion window size + Maximum segment size

After 1 round trip time, congestion window size = $(2)^1 = 2$ MSS

After 2 round trip time, congestion window size = $(2)^2 = 4$ MSS

After 3 round trip time, congestion window size = $(2)^3 = 8$ MSS and so on.

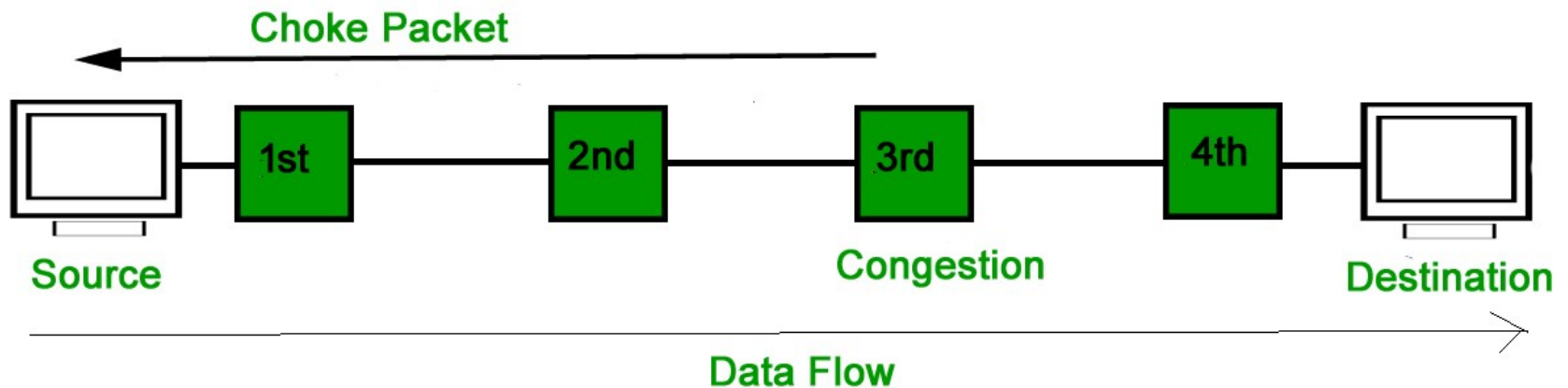


Choke Packet Technique:

Choke packet technique is applicable to both virtual networks as well as datagram subnets. A choke packet is a packet sent by a node to the source to inform it of congestion. Each router monitor its resources and the utilization at each of its output lines.

whenever the resource utilization exceeds the threshold value which is set by the administrator, the router directly sends a choke packet to the source giving it a feedback to reduce the traffic.

The intermediate nodes through which the packets has traveled are not warned about congestion.



Difference between Flow Control and Congestion Control

S.NO	FLOW CONTROL	CONGESTION CONTROL
1.	In flow control, Traffics are controlled which are flow from sender to a receiver.	In this, Traffics are controlled entering to the network.
2.	Data link layer and Transport layer handle it.	Network layer and Transport layer handle it.
3.	In this, Receiver's data is prevented from being overwhelmed.	In this, Network is prevented from congestion.
4.	In flow control, Only sender is responsible for the traffic.	In this, Transport layer is responsible for the traffic.
5.	In this, Traffic is prevented by slowly sending by the sender.	In this, Traffic is prevented by slowly transmitting by the transport layer.
6.	In flow control, buffer overrun is restrained in the receiver.	In congestion control, buffer overrun is restrained in the intermediate systems in the network.

Leaky Bucket Algorithm in Computer Networks

The leaky bucket algorithm is a method of temporarily storing a variable number of requests and organizing them into a set-rate output of packets.

Consider a bucket with a hole at the bottom. Whatever be the rate at which water enters into the bucket, it leaks out of the bucket at a constant rate through the hole.

The rate of flow is zero if there is no water in the bucket and if the bucket is full then the excess water spills over and it is lost.

This basic concept is applied in the case of the Leaky Bucket Algorithm which is nothing but a single server queuing system with constant service time.

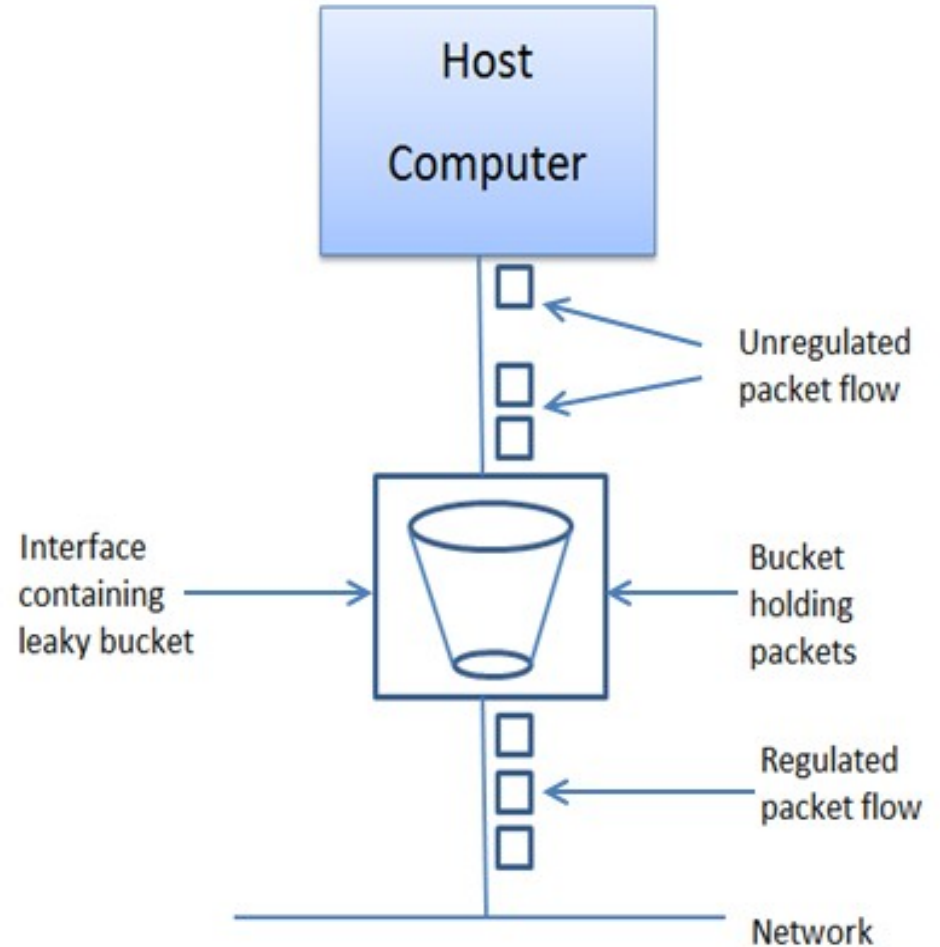
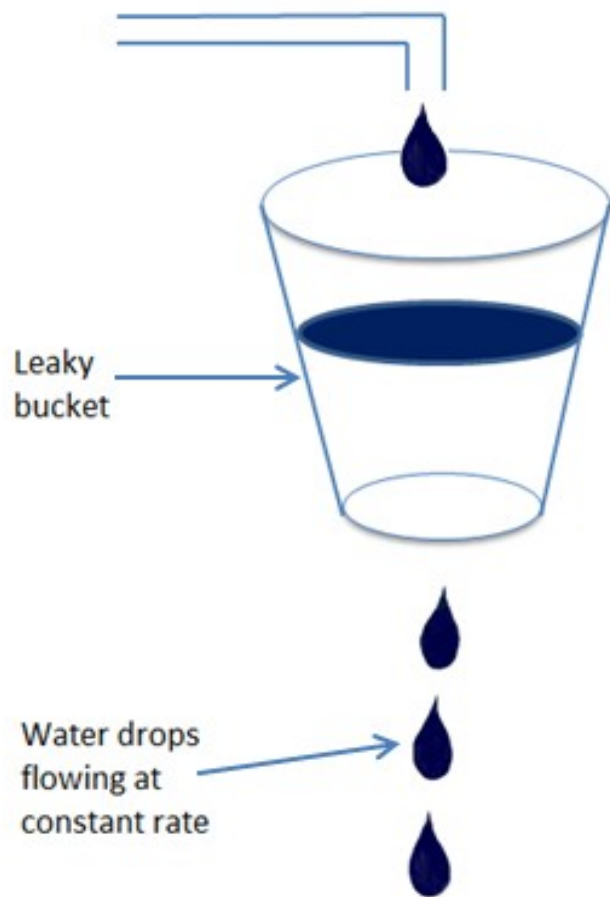


Fig: Leaky Bucket Algorithm

Basic Features of Leaky Bucket Algorithm:

1. Each host is connected to the network by an interface containing a leaky bucket that is a finite internal queue.
2. When space is available in a queue, a packet sent from an application is stored.
3. When the queue is full and a new packet is sent from an application is discarded.
4. This arrangement is built in the hardware or simulated by the host operating system.
5. As the packets are queued and released at regular interval with the regular amount and that reduces the chances of congestion.

Problem: As the packets carry an uneven amount of data, the actual loading effect generated by the packets on the network is also uneven.

Token Bucket Algorithm

It is an improved version of the leaky bucket algorithm, which can adjust flow in case of bursty traffic. It allows sending bursty traffic when required, up to the maximum size of the bucket.

As we know that, leaky bucket algorithm enforces a rigid pattern at the output stream, irrespective of the pattern of the input. For many applications, it is better to allow the output to speed up somewhat when a larger bursts arrives than to loose the data.

The token bucket algorithm allows idle hosts to accumulate credit for the future in the form of tokens. For each tick of the clock, the system sends n tokens to the bucket. The system removes one token for every cell (or byte) of data sent. The token bucket can easily be implemented with a counter. The token is initialised to zero.

Each time a token is added, the counter is incremented by 1. Each time a unit of data is sent, the counter is decremented by 1. When the counter is zero the host cannot send data.

This algorithm follows the following steps:

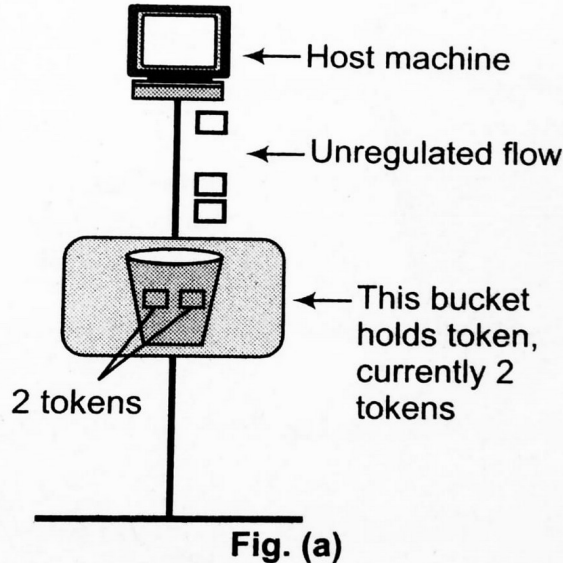
Step 1. In regular intervals tokens are thrown into the bucket.

Step 2. The bucket has a maximum capacity.

Step 3. If there is a ready packet, a token is removed from the bucket and the packet can be send.

Step 4. If there is no token in the bucket, the packet cannot be send.

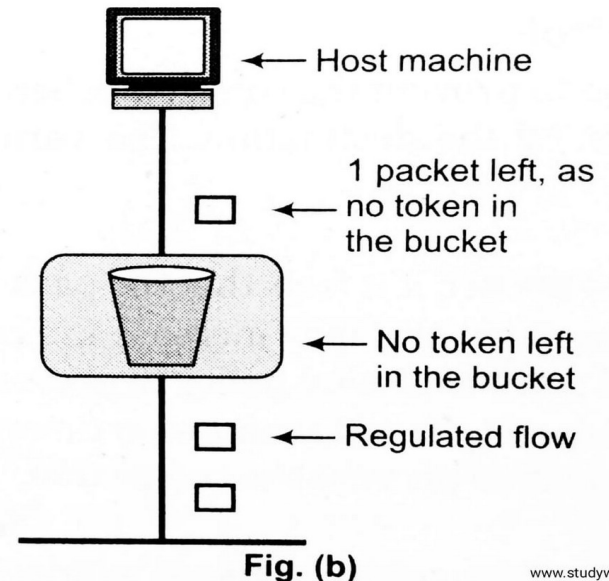
In Fig. (a), token bucket holding two tokens, before packets are send out.



by Mayank Rana

www.studywow.com

When token bucket after two packets are send. One packet still remains as no token is left. As shown in Fig(b).



by Mayank Rana

www.studywow.com

Now, if the host wants to send bursty data, it can consume all 10,000 tokens at once for sending 10,000 cells or bytes. Thus, a host can send bursty data as long as bucket is not empty. As shown in Fig(c).

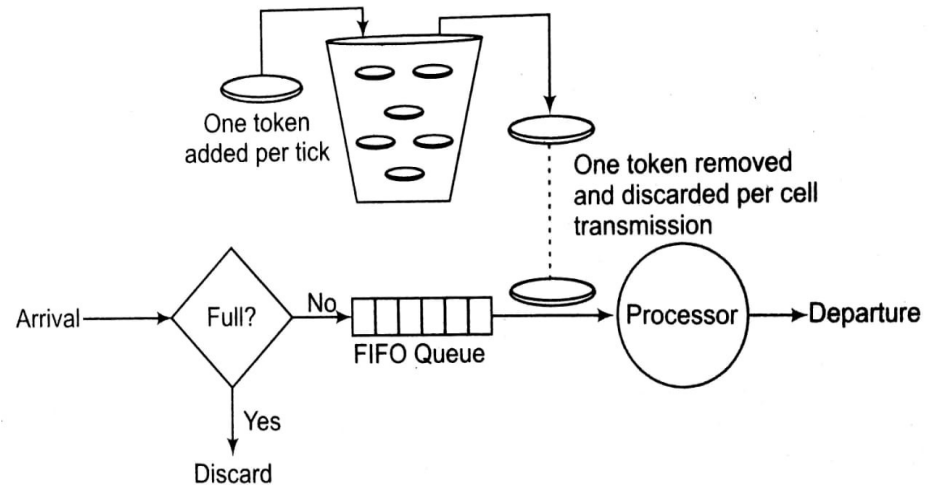


Fig. (c) Token bucket algorithm

by Mayank Rana

www.studywov.com

Some advantage of token Bucket over leaky bucket –

If bucket is full in token Bucket , tokens are discard not packets. While in leaky bucket, packets are discarded.

Token Bucket can send Large bursts at a faster rate while leaky bucket always sends packets at constant rate.

Difference between Leaky and Token buckets –

LEAKY BUCKET

When the host has to send a packet , packet is thrown in bucket.

Bucket leaks at constant rate

Bursty traffic is converted into uniform traffic by leaky bucket.

In practice bucket is a finite queue outputs at finite rate

TOKEN BUCKET

In this leaky bucket holds tokens generated at regular intervals of time.

Bucket has maximum capacity.

If there is a ready packet , a token is removed from Bucket and packet is send.

If there is a no token in bucket, packet can not be send.

Differences between TCP and UDP

TRANSMISSION CONTROL PROTOCOL (TCP)	USER DATAGRAM PROTOCOL (UDP)
TCP is a connection-oriented protocol. Connection-orientation means that the communicating devices should establish a connection before transmitting data and should close the connection after transmitting the data.	UDP is the Datagram oriented protocol. This is because there is no overhead for opening a connection, maintaining a connection, and terminating a connection. UDP is efficient for broadcast and multicast type of network transmission.
TCP is reliable as it guarantees delivery of data to the destination router.	The delivery of data to the destination cannot be guaranteed in UDP.
TCP provides extensive error checking mechanisms. It is because it provides flow control and acknowledgment of data.	UDP has only the basic error checking mechanism using checksums.
Sequencing of data is a feature of Transmission Control Protocol (TCP). this means that packets arrive in-order at the receiver.	There is no sequencing of data in UDP. If ordering is required, it has to be managed by the application layer.
TCP is comparatively slower than UDP.	UDP is faster, simpler and more efficient than TCP.
Retransmission of lost packets is possible in TCP, but not in UDP.	There is no retransmission of lost packets in User Datagram Protocol (UDP).
TCP has a (20-80) bytes variable length header.	UDP has a 8 bytes fixed length header.
TCP is heavy-weight.	UDP is lightweight.
TCP doesn't supports Broadcasting.	UDP supports Broadcasting.
TCP is used by HTTP, HTTPs, FTP, SMTP and Telnet.	UDP is used by DNS, DHCP, TFTP, SNMP, RIP, and VoIP.

QoS (quality of service)

Quality of service (QoS) refers to any technology that manages data traffic to reduce packet loss, latency and jitter on the network. QoS controls and manages network resources by setting priorities for specific types of data on the network.

QoS parameters

Organizations can measure QoS quantitatively by using several parameters, including the following:

Reliability, Lack of reliability means losing a packet or acknowledgement which entails retransmission.

Delay, Increase in delay means destination will find the packet later than expected, Importance of delay changes according to the various application.

Jitter, Variation of the delay is jitter, If the delay is not at a constant rate, it may result in poor quality.

Bandwidth, Increase in bandwidth means increase in the amount of data which can be transferred in given amount of time, Importance of bandwidth also varies according to various application.

Techniques for achieving good Quality of Service :

Over provisioning –

The logic of over provisioning is to provide greater router capacity, buffer space and bandwidth. It is an expensive technique as the resources are costly. Eg: Telephone System.

Buffering –

Flows can be buffered on the receiving side before being delivered. It will not affect reliability or bandwidth, but helps to smooth out jitter. This technique can be used at uniform intervals.

Traffic Shaping –

It is defined as about regulating the average rate of data transmission. It smooths the traffic on server side other than client side.

When a connection is set up, the user machine and subnet agree on a certain traffic pattern for that circuit called as *Service Level Agreement*. It reduces congestion and thus helps the carrier to deliver the packets in the agreed pattern.

Stream Control Transmission Protocol.(SCTP)

- It is a connection- oriented protocol in computer networks which provides a full-duplex association i.e., transmitting multiple streams of data between two end points at the same time that have established a connection in network.
- It is sometimes referred to as next generation TCP or TCPng, SCTP makes it easier to support telephonic conversation on Internet.
- A telephonic conversation requires transmitting of voice along with other data at the same time on both ends, SCTP protocol makes it easier to establish reliable connection.
- SCTP is also intended to make it easier to establish connection over wireless network and managing transmission of multimedia data.
- SCTP is a standard protocol (RFC 2960) and is developed by Internet Engineering Task Force (IETF).

Characteristics of SCTP :

❖ **Unicast with Multiple properties** –

It is a point-to-point protocol which can use different paths to reach end host.

❖ **Reliable Transmission** –

It uses SACK and checksums to detect damaged, corrupted, discarded, duplicate and reordered data. It is similar to TCP but SCTP is more efficient when it comes to reordering of data.

❖ **Message oriented** –

Each message can be framed and we can keep order of DataStream and tabs on structure. For this, In TCP, we need a different layer for abstraction.

❖ **Multi-homing** –

It can establish multiple connection paths between two end points and does not need to rely on IP layer for resilience.

Services Provided by SCTP

1. Multiple Streams:

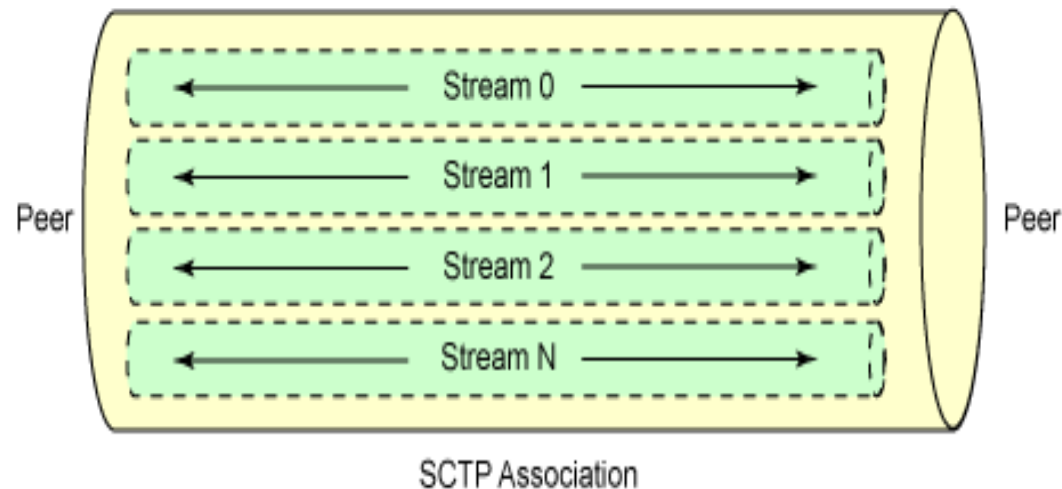
TCP is a stream-oriented protocol. Each connection between a TCP client and a TCP server involves one single stream.

The problem with this approach is that a loss at any point in the stream blocks the delivery of the rest of the data.

This can be acceptable when we are transferring text; it is not when we are sending real-time data such as audio or video.

SCTP allows multi stream service in each connection, which is called association in SCTP terminology.

If one of the streams is blocked, the other streams can still deliver their data.

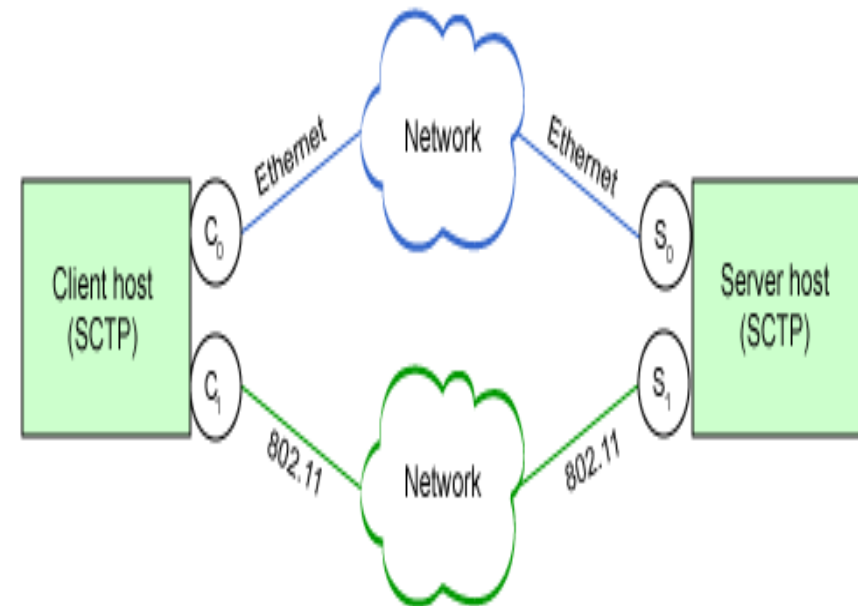
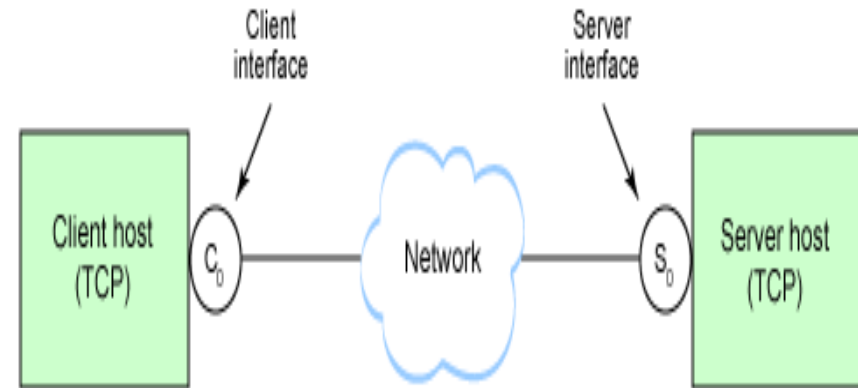


2. Multi homing:

A TCP connection involves one source and one destination IP address. This means that even if the sender or receiver is a multihomed host (connected to more than one physical address with multiple IP addresses), only one of these IP addresses per end can be utilized during the connection.

An SCTP association, on the other hand, supports multihoming service. The sending and receiving host can define multiple IP addresses in each end for an association. In this fault-tolerant approach, when one path fails, another interface can be used for data delivery without interruption.

This fault-tolerant feature is very helpful when we are sending and receiving a real-time payload such as Internet telephony.



3. Full-Duplex Communication:

Like TCP, SCTP offers full-duplex service, in which data can flow in both directions at the same time. Each SCTP then has a sending and receiving buffer, and packets are sent in both directions.

4. Connection-Oriented Service:

Like TCP, SCTP is a connection-oriented protocol. However, in SCTP, a connection is called an association. When a process at site A wants to send and receive data from another process at site B, the following occurs:

1. The two SCTPs establish an association between each other.
2. Data are exchanged in both directions.
3. The association is terminated.
4. **Reliable Service:** SCTP, like TCP, is a reliable transport protocol. It uses an acknowledgment mechanism to check the safe and sound arrival of data.

SCTP packet header Format

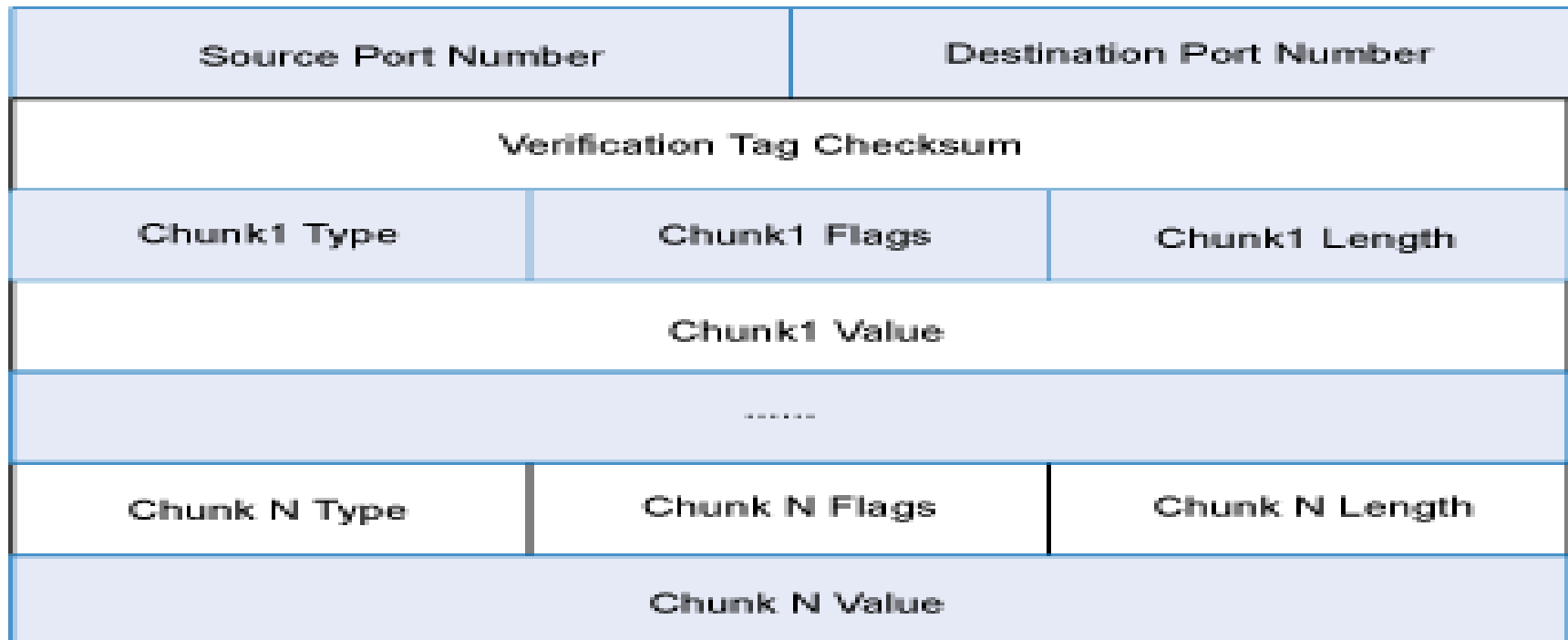
An SCTP packet has a mandatory general header and a set of blocks called chunks.

There are two types of chunks: control chunks and data chunks.

A control chunk controls and maintains the association.

A data chunk carries user data. In a packet, the control chunks come before the data chunks.

The following figure shows the general format of an SCTP packet.



Source port number	Identifies the sending port.
Destination port number	Identifies the receiving port..
Verification tag	Distinguishes stale packets from a previous connection. This is a 32-bit random value created during initialization.
Checksum	Uses the cyclic redundancy check (CRC32) algorithm to detect errors that might have been introduced during data transmission.

Chunk Type	Identifies the contents of the chunk value field. This is 1- byte long.
Chunk Flags	Consists of 8 flag-bits whose definition varies with the chunk type. The default value is zero.
Chunk Length	Specifies the total length of the chunk in bytes. This field is 2 - bytes long. If the chunk does not form a multiple of 4 bytes it is implicitly padded with zeros which are not included in the chunk length.
Chunk Value	A general purpose data field.

Advantages of SCTP :

- It is a full- duplex connection i.e. users can send and receive data simultaneously.
- It allows half- closed connections.
- The message's boundaries are maintained and application doesn't have to split messages.
- It has properties of both TCP and UDP protocol.

Disadvantages of SCTP :

- One of key challenges is that it requires changes in transport stack on node.
- Applications need to be modified to use SCTP instead of TCP/UDP.
- Applications need to be modified to handle multiple simultaneous streams.

Sockets

Sockets allow communication between two different processes on the same or different machines.

To be more precise, it's a way to talk to other computers using standard Unix file descriptors.

In Unix, every I/O action is done by writing or reading a file descriptor. A file descriptor is just an integer associated with an open file and it can be a network connection, a text file, a terminal, or something else.

The Socket Interface

- Client and server use the transport protocol to communicate.
- When it interacts with protocol, an application must specify :whether it is a server or a client (that is, whether it will wait passively or actively initiate communication)
- In addition, the sender must specify the data to be sent, and the receiver must specify where incoming data should be placed.

The interface used by an application is known as an Application Program Interface (API).

- An API defines a set of operations that an application can perform when interacting with protocol software and details such as arguments required.
- Usually an API contains a separate procedure for each logical function

TCP Sockets

TCP Sockets (or virtual ports) are used in TCP (and UDP) communication to identify unique end-to-end connections.

They are called 'virtual ports' because a single physical connector can serve multiple connections.

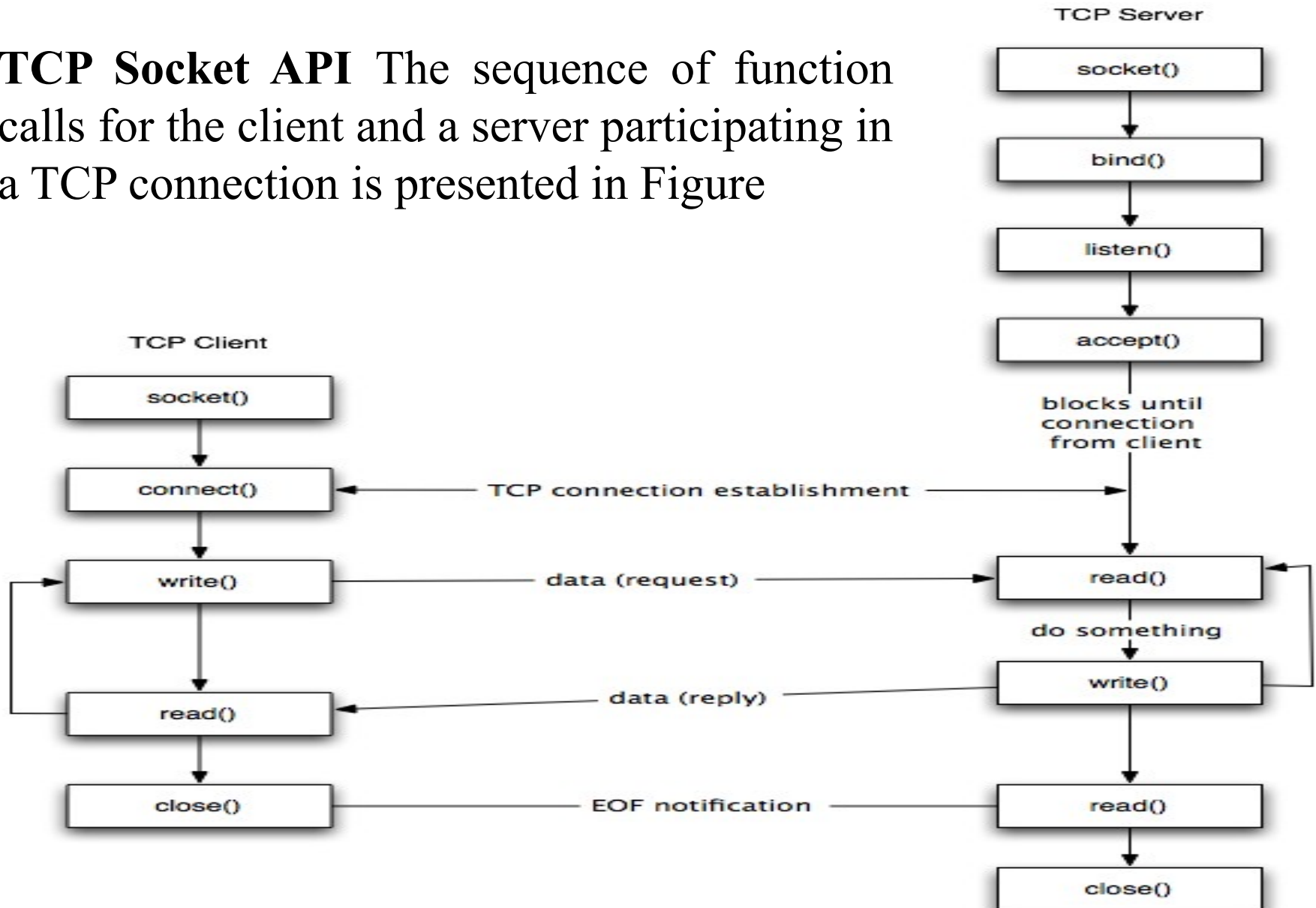
Each side of a socket connection uses its own port number, which does not change during the life of that connection.

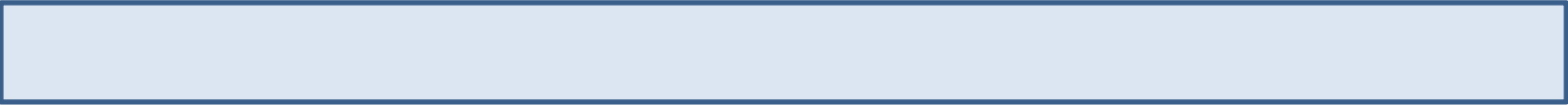
The port number and IP address together uniquely identify an endpoint. Together, two endpoints are considered a 'socket'.

Sockets Primitives

Primitives	Meaning
SOCKET	Create a New Communication Endpoint.
BIND	Attach a Local Address to a SOCKET.
LISTEN	Shows the Willingness to Accept Connections.
ACCEPT	Block the Caller until a Connection Attempts Arrives.
CONNECT	Actively Attempt to Establish a Connection.
SEND	Send Some Data over Connection.
RECEIVE	Receive Some Data from the Connection.
CLOSE	Release the Connection.

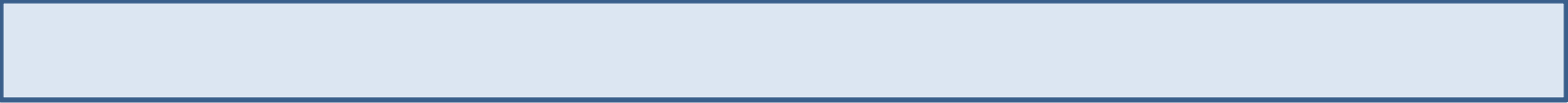
TCP Socket API The sequence of function calls for the client and a server participating in a TCP connection is presented in Figure





As shown in the figure, the steps for establishing a TCP socket on the client side are the following:

- Create a socket using the `socket()` function;
- Connect the socket to the address of the server using the `connect()` function;
- Send and receive data by means of the `read()` and `write()` functions.
- Close the connection by means of the `close()` function.



The steps involved in establishing a TCP socket on the server side are as follows:

- Create a socket with the `socket()` function;
- Bind the socket to an address using the `bind()` function;
- Listen for connections with the `listen()` function;
- Accept a connection with the `accept()` function system call.

This call typically blocks until a client connects with the server.

- Send and receive data by means of `send()` and `receive()`.
- Close the connection by means of the `close()` function.

UDP Socket API

Figure shows the interaction between a UDP client and server.

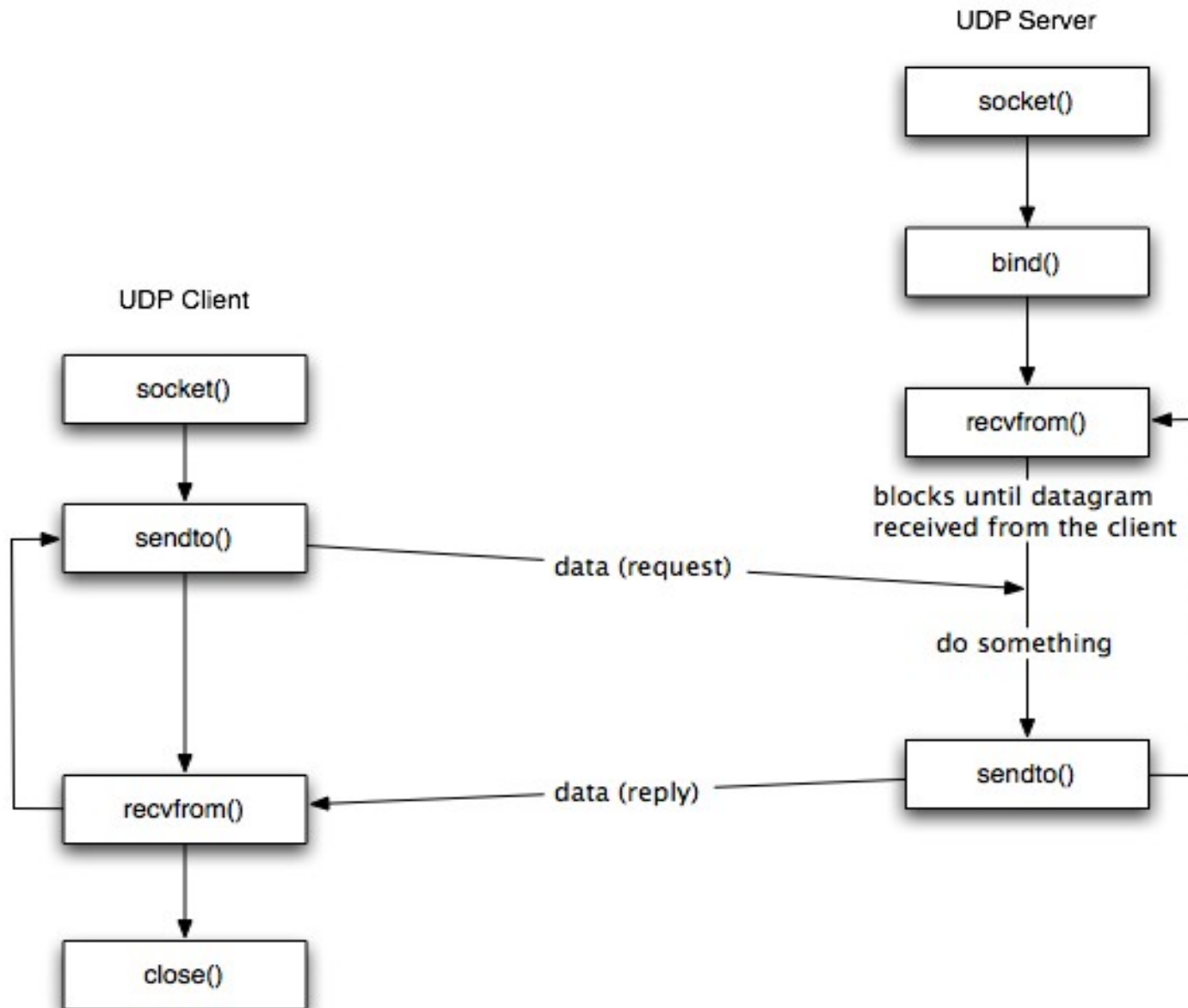
First of all, the client does not establish a connection with the server.

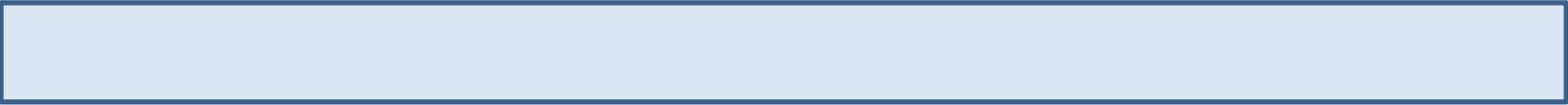
Instead, the client just sends a datagram to the server using the `sendto` function which requires the address of the destination as a parameter.

Similarly, the server does not accept a connection from a client.

Instead, the server just calls the `recvfrom` function, which waits until data arrives from some client.

`recvfrom` returns the IP address of the client, along with the datagram, so the server can send a response to the client.





As shown in the Figure, the steps of establishing a UDP socket communication on the client side are as follows:

Create a socket using the `socket()` function;

Send and receive data by means of
the `recvfrom()` and `sendto()` functions.

The steps of establishing a UDP socket communication on the server side are as follows:

Create a socket with the `socket()` function;

Bind the socket to an address using the `bind()` function;

Send and receive data by means of `recvfrom()` and `sendto()`.